

1. Scop si problema

Aplicatia modeleaza un sistem de inferenta genealogica: porneste de la fapte de baza (mame-copii, sot-sotie) si deduce relatii suplimentare (frati, legaturi copil-parinte complete, bunici, veri). Reguli si verificari sunt implementate procedural, nu printr-un motor declarativ. Problema centrala este extinderea bazei de cunostinte cu relatii colaterale, evitand duplicatele si pastrand perechile simetrice in ordine alfabetica pentru consistenta.

2. Reprezentarea cunostintelor

Faptele sunt structurate in clase simple.

- Mama (relatie mama-copil)
- Frate (perechi ordonate de frati)
- Sot (cupluri sot-sotie, fara distinctie de gen)
- Copil (relatie copil-parinte generica)
- Bunic (relatie bunic-nepot)
- Ver (perechi ordonate de veri)

Dimensiunea maxima a bazei este limitata prin `MAX_FAPTE = 100`. In `main` se aloca vectori pentru fiecare tip de fapt si contoare asociate (init pe zero), formand baza operationala in memorie.

3. Faptele initiale

Baza factuala este incarcata manual:

- Relatii mama-copil: de exemplu Roberta→Costin, Roberta→Ana, Cosmina→Andrei, Cosmina→Cristina, etc.
 - Relatii sot-sotie: de exemplu Roberta–Gheorghe, Cosmina–Catalin, Maria–Petru, Lidia–Sorin, Sorina–Mihai, Elena–Dan, Nora–Danut, Irina–Tudor, Olga–Mircea.
- Aceste fapte sunt considerate adevaruri de baza; deducerile ulterioare se bazeaza exclusiv pe ele.

4. Reguli de inferenta si ratioment

Programul aplica patru reguli, in ordine, pentru a extinde baza.

4.1. Frati

- Premisa: doi copii cu aceeași mama sunt frati.

- Implementare: se parcurg toate perechile de fapte `Mama` ; se ordoneaza alfabetic perechea (a, b) si se verifica daca exista deja in lista `Frate` . Daca nu, se adauga.
- Scop: eliminarea simetriilor (A-B vs B-A) si a dublurilor.

4.2. Copil-parinte complet

- Premisa: din fiecare fapt `Mama` , se adauga copil→mama; daca mama are partener in `Sot` , se adauga copil→tata.
- Implementare: se foloseste cautarea partenerului; se evita adaugarea unui parinte inexistent (cand nu se gaseste partener).
- Scop: completarea ambelor ramuri parentale acolo unde exista cuplu declarat.

4.3. Bunici

- Premisa: mama unei mame este bunica, iar partenerul acelei bunici devine bunic.
- Implementare: pentru fiecare relatie mama-copil, se cauta intrarea unde mama apare ca si copil; se adauga bunic-nepot si, daca exista partener al bunicii, si acesta este adaugat. Se verifica dublurile pentru fiecare insertie.
- Scop: propagarea in sus pe linia maternală si lateral prin cuplu.

4.4. Veri

- Premisa: doi copii sunt veri daca parintii lor sunt frati intre ei.
- Implementare: pentru fiecare pereche de copii cu parinti diferiti, se verifica daca parintii formeaza o pereche in lista `Frate` ; daca da, se ordoneaza alfabetic perechea de veri si se insereaza doar daca nu exista.
- Scop: generarea relatiilor colaterale intre generatii, cu consistenta ordonarii.

5. Flux de executie

1. Initializare vectori si contoare in `main` .
2. Incarcare fapte brute (mame-copii si sotii) in baza in memorie.
3. Aplicare regula pentru frati.
4. Completare relatii copil-parinte cu ajutorul cuplurilor.
5. Deducere bunici (linia materna + partenerul bunicii).
6. Deducere veri (copii ai parintilor care sunt frati).
7. Afisare rezultate: copii, frati, bunici, veri.

Ordinea este esentiala: fratii trebuie calculati inainte de veri, iar copiii completati inainte de bunici si veri.

6. Functii de utilitate si output

- `gasestePartener` cauta intr-o lista de cupluri si intoarce partenerul unei persoane sau sirul vid daca nu exista.
- Functii de afisare (`afiseazaCopii` , `afiseazaFrati` , `afiseazaBunici` , `afiseazaVeri`) listeaza relatiile obtinute, oferind transparenta asupra bazei extinse.

7. Corectitudine si controlul duplicatelor

- Perechile simetrice (frati, veri) sunt ordonate alfabetic pentru a reduce variantele A-B si B-A la o singura forma canonica.
- Inainte de inserare se verifica explicit existenta relatiei; se evita dublurile la frati, bunici, veri.
- Pentru copii-parinti, nu se insereaza al doilea parinte daca `gasestePartener` nu gaseste un cuplu.

8. Complexitate

- Frati: $O(n^2)$ in numarul de fapte `Mama` (comparare perechi).
- Copil-parinte: $O(n)$ in `Mama` cu cautare partener $O(m)$; in cel mai rau caz $O(n*m)$.
- Bunici: $O(n^2)$ in `Mama` (cautarea mamei mamei).
- Veri: $O(c^2 * f)$, unde c este numarul de copii dedusi, f numarul de perechi de frati.

9. Concluzie

Programul exemplifica un lant procedural de inferenta genealogica: pleaca de la fapte de baza si aplica reguli deterministe pentru a extinde baza de cunostinte. Structurarea pe clase simple, ordonarea perechilor si verificarea dublurilor produc o harta coerenta de relatii (frati, parinti, bunici, veri).